Listing 11.4 shows the code for the `create_plan` function from the `BasicNexusPlanner` class; tools such as LangChain and SK use similar patterns. The process loads the agent's actions as a string. The goal and available functions list are then inserted into the planner prompt template using the `PromptTemplateManager`, which is just a wrapper for the template-handling code. Template handling is done with simple regex but can also be more sophisticated using tools such as Jinja2, Handlebars, or Mustache.

Listing 11.4 `basic_nexus_planner.py` (`create_plan`)

```
def create_plan(self, nexus, agent, goal: str, prompt: str = PROMPT) -> Plan:
        selected_actions = nexus.get_actions(agent.actions)
        available_functions_string = "\n\n".join(
            format_action(action) for action in selected_actions
        )     #1

        context = {}     #2
        context["goal"] = goal
        context["available_functions"] = available_functions_string

        ptm = PromptTemplateManager()     #3
        prompt = ptm.render_prompt(prompt, context)

        plan_text = nexus.execute_prompt(agent, prompt)     #4
        return Plan(prompt=prompt,
                    goal=goal,
                    plan_text=plan_text)     #5
```

#1 Loads the agent's available actions and formats the result string for the planner
#2 The context will be injected into the planner prompt template.
#3 A simple template manager, similar in concept to Jinja2, Handlebars, or Mustache
#4 Sends the filled-in planner prompt to the LLM
#5 The results (the plan) are wrapped in a Plan class and returned for execution.

The code to execute the plan, shown in listing 11.5, parses the JSON string and executes the functions. When executing the plan, the code detects the particular `for-each` function, which iterates through a list and executes each element in a function. The results of each function execution are added to the context. This context is passed to each function call and returned as the final output.